

Document number	P2724R0
Date	2022-12-11
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Evolution Working Group (EWG)

constant dangling

Table of contents

- constant dangling
 - Changelog
 - Abstract
 - Motivation
 - Motivating examples
 - Proposed Wording
 - In Depth Rationale
 - Value Categories
 - Performance Considerations
 - Tooling Opportunities
 - Summary
 - Frequently Asked Questions
 - References

Changelog

R0

- The constant content was extracted and merged from the `temporary storage class` specifiers ^[1] and `implicit constant initialization` ^[2] proposals.

Abstract

This paper proposes the standard adds anonymous global constants to the language with the intention of automatically fixing a shocking type of dangling which occurs when constants or that which should be constants dangle. This is shocking because constant like instances should really have constant-initialization meaning that they should have static storage duration and consequently should not dangle. This trips up beginner code requiring teaching dangling on day one. It is annoying to non beginners. Constants are used as defaults in production code. Constants are also frequently used in test and example code. Further, many instances of dangling used by non c++ language comparisons frequently use constants as examples.

Motivation

There are multiple resolutions to dangling in the c++ language.

1. Produce an error
 - `Simpler implicit move` [3]
2. Fix with block/variable scoping
 - `Fix the range-based for loop, Rev2` [4]
 - `Get Fix of Broken Range-based for Loop Finally Done` [5]
3. Fix by making the instance global
 - **This proposal**

All are valid resolutions and individually are better than the others, given the scenario. This proposal is focused on the third option, which is to fix by making the instance global.

Dangling the stack is shocking because it violates our trust in our compilers and language, since they are primarily responsible for the stack. However, there are three types of dangling that are even more shocking than the rest.

1. Returning a **direct** reference to a local
 - partially resolved by `Simpler implicit move` [3:1]
2. Immediate dangling
3. **Dangling Constants**

Making an instance global is a legitimate fix to dangling.

C++ Core Guidelines

F.43: Never (directly or indirectly) return a pointer or a reference to a local object ^[6]

Reason *To avoid the crashes and data corruption that can result from the use of such a dangling pointer.* ^[6:1]

...

Note *This applies only to non-static local variables. All static variables are (as their name indicates) statically allocated, so that pointers to them cannot dangle.* ^[6:2]

While making an instance global doesn't fix all dangling in the language, it is the only resolution that can fix all three most shocking types of dangling provided the instance in question is a constant. It is also the **best** fix for these instances.

Since `constexpr` was added to the language in C++11 there has been an increase in the candidates of temporary instances that could be turned into global constants. ROMability was in part the motivation for `constexpr` but the requirement was never made. Even if a C++ architecture doesn't support ROM, it is still required by language to support `static storage duration` and `const`. Matter of fact, due to the immutable nature of constant-initialized constant expressions, these expressions/instances are constant for the entire program even though they, at present, don't have `static storage duration`, even if just logically. There is a greater need now that more types are getting `constexpr` constructors. Also types that would normally only be dynamically allocated, such as `string` and `vector`, since C++20, can also be `constexpr`. This has opened up the door wide for many more types being constructed at compile time.

Motivating Examples

Before diving into the examples, let's discuss what exactly is being asked for. There are two features; one implicit and the other explicit.

implicit constant initialization

If a temporary argument is constant-initialized ^[7] (7.7 *Constant expressions [expr.const]*) and its argument/instance type is a `LiteralType` and its parameter/local/member type is `const` and not `mutable` then the instance is implicitly created with `constant initialization`.

As such it has `static storage duration` and can't dangle.

explicit constant initialization

The `constinit` specifier can be applied to temporaries. Applying it asserts that the temporary was `const-initialized`, that the argument type is a `LiteralType` and its parameter/local/member type is `const` and not `mutable`. This explicitly gives the temporary static storage duration.

While implicit constant initialization automatically fixes dangle, `constinit` allows the programmers to manually and explicitly fix some dangling. The former is better for programmers and the language, while the later favors code reviewers or programmers who copy an example and want to have the compiler, momentarily, verify whether it is correct.

So what sorts of dangling does this fix for us. Besides fixing some dangling, this also fixes some inconsistencies between string literals ^[7:1] (5.13.5 String literals *[lex.string]*) and other literal types.

```
// does not dangle because string literals are lvalues with static storage duration
std::string_view sv = "hello world";
// immediate dangling reference
// even though "hello world"s is a constant-initialized,
// literal type that is only being used in a const fashion
std::string_view sv = "hello world"s; // fixed in this proposal
std::string_view sv = constinit "hello world"s; // fixed in this proposal
```

This is reasonable based on how programmers reason about constants being immutable variables and temporaries which are known at compile time and do not change for the life of the program. This also works with plain old references.

```
struct X
{
    int a, b;
};

const int& get_a(const X& x)
{
    // sporadically dangling, returning indirect reference
    return x.a;
}

const int& a = get_a({4, 2});
a; // currently dangling, will be 4 with this proposal
```

“Such a feature would also help to ... fix several bugs we see in practice:” ^[8]

“Consider we have a function returning the value of a map element or a default value if no such element exists without copying it:” ^[8:1]

```
const V& findOrDefault(const std::map<K,V>& m, const K& key, const V& defvalue);
```

"then this results in a classical bug:" [8:2]

```
std::map<std::string, std::string> myMap;  
const std::string& s = findOrDefault(myMap, key, "none"); // runtime bug if key no
```

Is this really a bug? With this proposal, it isn't! Here is why. The function `findOrDefault` **expects** a `const string&` for its third parameter. Since C++20, `string`'s constructor is `constexpr`. It **CAN** be constructed as a constant expression. Since all the arguments passed to this `constexpr` constructor are constant expressions, in this case `"none"`, the temporary `string defvalue` **IS** also constant-initialized [7:2] (7.7 Constant expressions [expr.const]). This paper advises that if you have a non mutable `const` that it is constant-initialized, that the variable or temporary undergoes constant initialization [7:3] (6.9.3.2 Static initialization [basic.start.static]). In other words it has implicit static storage duration. The temporary would actually cease to be a temporary. As such this usage of `findOrDefault` **CAN'T** dangle.

The pain of immediate dangling associated with temporaries are especially felt when working with other anonymous language features of C++ such as lambda functions and coroutines.

Lambda functions

Whenever a lambda function captures a reference to a temporary it immediately dangles before an opportunity is given to call it, unless it is a immediately invoked lambda/function expression.

```
[&c1 = "hello"s](const std::string& s)// OK  
{  
    return c1 + " "s + s;  
}("world"s); // immediately invoked lambda/function expression  
  
auto lambda = [&c1 = "hello"s](const std::string& s) // immediate dangling  
{  
    return c1 + " "s + s;  
}  
// ...  
lambda("world"s);
```

This problem is resolved when the scope of temporaries has static storage duration instead of the containing expression provided `c1` resolves to a `const std::string&` since `c1` was constant-initialized. The `constexpr` specifier could ensure this.

Coroutines

Similarly, whenever a coroutine gets constructed with a reference to a temporary it immediately dangles before an opportunity is given for it to be `co_await` ed upon.

```
generator<char> each_char(const std::string& s) {
    for (char ch : s) {
        co_yield ch;
    }
}

int main() {
    auto ec = each_char("hello world");// immediate dangling
    for (char ch : ec) {
        std::print(ch);
    }
}
```

This specific immediately dangling example is fixed by implicit constant initialization since the parameter `s` expects a `const std::string&` and it was constant-initialized.

It should be noted too that the current rules of temporaries discourages the use of temporaries because of the dangling it introduces. However, if the lifetime of temporaries was increased to a reasonable degree than programmers would use temporaries more. This would reduce dangling further because there would be fewer named variables that could be propagated outside of their containing scope. This would also improve code clarity by reducing the number of lines of code allowing any remaining dangling to be more clearly seen.

Proposed Wording

6.7.5.4 Automatic storage duration [basic.stc.auto]

¹ Variables that belong to a block or parameter scope and are not explicitly declared static, `thread_local`, `extern` or had not underwent implicit constant initialization (6.9.3.2) have automatic storage duration. The storage for these entities lasts until the block in which they are created exits.

...

6.9.3.2 Static initialization [basic.start.static]

...

² Constant initialization is performed explicitly if a variable or temporary object with static or thread storage duration is constant-initialized (7.7). Constant initialization is performed implicitly if a non mutable const variable or non mutable const temporary object is constant-initialized (7.7). If constant initialization is not performed, a variable with static storage duration (6.7.5.2) or thread storage duration (6.7.5.3) is zero-initialized (9.4). Together, zero-initialization and constant initialization are called static initialization; all other initialization is dynamic initialization. All static initialization strongly happens before (6.9.2.2) any dynamic initialization.

...

9.2.7 The `constinit` specifier [`dcl.constinit`]

¹ If the `constinit` specifier is applied to a temporary, it gives the temporary static storage duration, asserts that the argument is a `LiteralType` and asserts that the parameter type is not mutable and `const` otherwise the `constinit` specifier shall be applied only to a declaration of a variable with static or thread storage duration. If the specifier is applied to any declaration of a variable, it shall be applied to the initializing declaration. No diagnostic is required if no `constinit` declaration is reachable at the point of the initializing declaration.

...

NOTE: Wording still need to capture that these temporaries are no longer temporaries and that their value category is `lvalue` .

In Depth Rationale

There is a general expectation across programming languages that constants or more specifically constant literals are “immutable values which are known at compile time and do not change for the life of the program”. ^[9] In most programming languages or rather the most widely used programming languages, constants do not dangle. Constants are so simple, so trivial (English wise), that it is shocking to even have to be conscience of dangling. This is shocking to `c++` beginners, expert programmers from other programming languages who come over to `c++` and at times even shocking to experienced `c++` programmers.

There is already significant interest in this type of feature from programmers. Just look at `c23` as an example. For instance, the `Introduce storage-class specifiers for compound literals` ^[10] and The `'constexpr'` specifier ^[11] allows `C` programmers to specify `static` , `constexpr` and `thread_local` as storage class specifiers on their compound literals. The compound literals equivalent in `c++` is `LiteralType` and temporaries. This paper reuses our existing keyword `constinit` over `static` because of what we all know from the `c++` Core Guidelines ^[12].

I.2: Avoid non-const global variables^[12:1]

Reason Non-const global variables hide dependencies and make the dependencies subject to unpredictable changes.^[12:2]

I also did not choose `constexpr`, though that may be better for greater C compatibility, since I wrote my proposals before my seeing the C paper. Also `constinit` better matches that which these features are doing in the context of existing C++ terminology of constant initialization. Further, there are differences in what `constexpr` means to C++ and C, at present.

It should also be noted that these concepts are already in the standard just not fully exposed in the language. For instance, string literals already have static storage duration and attempting to modify one is undefined.

Working Draft, Standard for Programming Language C++ [7:4]

“5.13.5 String literals [lex.string]”

“⁹ **Evaluating a string-literal results in a string literal object with static storage duration** (6.7.5). *Whether all string-literals are distinct (that is, are stored in nonoverlapping objects) and whether successive evaluations of a string-literal yield the same or a different object is unspecified.*”

“[Note 4: The effect of attempting to modify a string literal object is undefined. — end note]”

Further, this behavior happens all the time with evaluations of constant expressions but unfortunately we can't enjoy all the benefits thereof.

“6.9.3.2 Static initialization [*basic.start.static*]”

“₁ Variables with static storage duration are initialized as a consequence of program initiation. Variables with thread storage duration are initialized as a consequence of thread execution. Within each of these phases of initiation, initialization occurs as follows.”

“₂ Constant initialization is performed if a variable or temporary object with static or thread storage duration is constant-initialized (7.7). ...”

...

“₃ An implementation is permitted to perform the initialization of a variable with static or thread storage duration as a static initialization even if such initialization is not required to be done statically, provided that ...”

These ROM-able instances do not dangle as globals but from our code perspective they currently look like dangling locals. This causes false positive with our static analyzers and the programmer's themselves. If we would just admit from a language standpoint that these are indeed constants than not only do we fix some dangling but also our mental model. This same reference also says “constant initialization is performed if a ... temporary object with static ... storage duration is constant-initialized”. Programmers can't fully utilize this scenario because at present we can only use `static` on class members and locals but not temporary arguments. Since the code identified by this paper is already subject to constant initialization than there is no real chance of these changes causing any breakage.

Value Categories

If some temporaries can be changed to have global scope than how does it affect their value categories? Currently, if the literal is a string than it is a `lvalue` and it has global scope. For all the other literals, they tend to be a `prvalue` and have statement scope.

	movable	unmovable
named	xvalue	lvalue
unnamed	prvalue	?

From the programmers perspective, global temporaries are just anonymously named variables. When they are passed as arguments, they have life beyond the life of the function that it is given to. As such the expression is not movable. As such, the desired behavior described throughout the paper is that they are `lvalues` which makes sense from a anonymously named standpoint. However, it must be said that technically they are unnamed which places them into the value category that `c++` currently does not have; the unmovable unnamed. The point is, this is simple whether it is worded as a `lvalue` or an unambiguous new value category that behaves like a `lvalue`. Regardless of which, there are some advantages that must be pointed out.

Avoids superfluous moves

The proposed avoids superfluous moves. Copying pointers and lvalue references are cheaper than performing a move which is cheaper than performing any non trivial value copy.

Undo forced naming

The proposed makes using types that delete their `rvalue` reference constructor easier to use. For instance, `std::reference_wrapper` can't be created/reassigned with a `rvalue` reference, i.e. temporaries. Rather, it must be created/reassigned with a `lvalue` reference created on a seperate line. This requires superfluous naming which increases the chances of dangling. Further, according to the `C++ Core Guidelines`, it is developers practice to do the following:

- *ES.5: Keep scopes small* [^{cppcg5}]
- *ES.6: Declare names in for-statement initializers and conditions to limit scope* [^{cppcg6}]

```

// currently not permitted; works as proposed
std::reference_wrapper<int> rwi1(5);
// current forced usage
int value1 = 5;
std::reference_wrapper<int> rwi2(value1);
if(randomBool())
{
    int value2 = 7;// ES.5, ES.6
    rwi2 = ref(value2);// dangles with block scope
    rwi2 = ref(7);// ok, safe and easy with global scope proposal
    rwi2 = 7;// might make sense to add back direct assignment operator
}
else
{
    int value3 = 9;// ES.5, ES.6
    rwi2 = ref(value3);// dangles with block scope
    rwi2 = ref(9);// ok, safe and easy with global scope proposal
    rwi2 = 9;// might make sense to add back direct assignment operator
}

```

Since the variable `value2` and `value3` is likely to be created manually at block scope instead of variable scope, it can accidentally introduce more dangling. Constructing and reassigning with a global scoped `lvalue` temporary avoids these common dangling possibilities along with simplifying the code.

Performance Considerations

There are at least three ways to provide a non dangling globalish constant.

- ROM i.e. hardware
- `const` and `static` i.e. c++ language
- assembly opcode with inline constant i.e. machine code level

While the first two are addressable, the last one isn't.

In the next three examples, the same assembly is produced regardless of whether the literal `5` was provided via a native literal, a `constexpr` or a `const` global. The following results were produced in Compiler Explorer (<https://godbolt.org/>) using both “x86-64 clang (trunk) -std=c++20 -O3” and “x86-64 gcc (trunk) -std=c++20 -O3”.

values that are [logically] global constants

local constant but logically a global constant

```
int main()
{
    return 5;
}
```

```
main:                                     # @main
    mov     eax, 5
    ret
```

constant expression i.e. logically a global constant

```
constexpr int return5()
{
    return 5;
}
```

```
int main()
{
    return return5();
}
```

```
main:                                     # @main
    mov     eax, 5
    ret
```

an actual global

```
const int GLOBAL = 5;
```

```
int main()
{
    return GLOBAL;
}
```

```
main:                                     # @main
    mov     eax, 5
    ret
```

The point is all three are logically non dangling, constant global. Now let's look at reference examples.

immediate dangling

Not only do all three following examples produce the exact same assembly, they also provide the exact same assembly as the previous three examples. They are all essentially global constants from the assembly and programmer standpoint but the current standard says two of the three dangle, unnecessarily.

local constant but logically a global constant

```
int main()
{
    const int& reflocal = 5; // immediate dangling
    return reflocal;
}
```

```
main:                                     # @main
    mov     eax, 5
    ret
```

```
int main()
{
    const int local = 5;
    const int& reflocal = local;
    return reflocal;
}
```

```
main:                                     # @main
    mov     eax, 5
    ret
```

constant expression i.e. logically a global constant

```
constexpr int return5()
{
    return 5;
}

int main()
{
    const int& rlocal = return5();// immediate dangling
    return rlocal;
}
```

```
main:                                     # @main
    mov     eax, 5
    ret
```

an actual global

```
const int GLOBAL = 5;

int main()
{
    const int& rlocal = GLOBAL;
    return rlocal;
}
```

```
main:                                     # @main
    mov     eax, 5
    ret
```

indirect dangling of caller's local

Similarly to, the next three examples produce the same assembly in the 3 `clang` cases and 2 of the `gcc` cases. `gcc` would have produced the same result in its 2nd case had it had treated the `const` expected evaluation of a constant expression as a global constant as its third case did.

local constant but logically a global constant

```
const int& potential_dangler(const int& passthrough)
{
    return passthrough;
}
```

```
int main()
{
    const int local = 5;
    const int& reflocal = potential_dangler(local);
    return reflocal;
}
```

```
potential_dangler(int const&):                # @potential_dangler(int const&)
    mov     rax, rdi
    ret

main:                                         # @main
    mov     eax, 5
    ret
```

constant expression i.e. logically a global constant

```
constexpr int return5()
{
    return 5;
}
```

```
const int& potential_dangler(const int& passthrough)
{
    return passthrough;
}
```

```
int main()
{
    const int& reflocal = potential_dangler(return5());
    return reflocal;
}
```

x86-64 clang (trunk) -std=c++20 -O3

[illegible]

x86-64 gcc (trunk) -std=c++20 -O3

NOTE: Can't really say what GCC is doing with the `xor` . However, if GCC had treated the resolved constant expression which is `const` required as a `const` global as in the next example than the results would have been the same.

```
potential_dangler(int const&):
    mov     rax, rdi
    ret
main:
    xor     eax, eax
    ret
```

an actual global

```
const int GLOBAL = 5;

const int& potential_dangler(const int& passthrough)
{
    return passthrough;
}

int main()
{
    const int& reflocal = potential_dangler(GLOBAL);
    return reflocal;
}
```

[illegible]

In all these logically global constant cases, no instance was actually stored global but was perfectly inlined as an assembly opcode constant. So, the worst case performance of this proposal would be a single upfront load time cost. Contrast that with the current potential local constant cost of constantly creating and destroying instances, even multiple times concurrently in different threads. Even the proposed cost can go from 1 to 0 while the current non global local could result in superfluous dynamic allocations since `std::string` and `std::vector` are now `constexpr`.

Microsoft's compiler and existing dangling detection

Things really get interesting when we factor Microsoft's compiler into the equation and contrast its dangling detection between optimized configurations.

x64 msvc v19.latest

indirect dangling of caller's local

temporary constant but logically a global constant

```
const int& potential_dangler(const int& passthrough)
{
    return passthrough;
}

int main()
{
    const int& reftemp = potential_dangler(5);
    return reftemp;
}
```

/Ox optimizations

```
passthrough$ = 8
; potential_dangler
int const & potential_dangler(int const &) PROC
    mov     QWORD PTR [rsp+8], rcx
    mov     rax, QWORD PTR passthrough$[rsp]
    ret     0
; potential_dangler
int const & potential_dangler(int const &) ENDP

$T1 = 32
reftemp$ = 40
main PROC
$LN3:
    sub     rsp, 56; 00000038H
    mov     DWORD PTR $T1[rsp], 5
    lea     rcx, QWORD PTR $T1[rsp]
    ; potential_dangler
    call    int const & potential_dangler(int const &)
    mov     QWORD PTR reftemp$[rsp], rax
    mov     rax, QWORD PTR reftemp$[rsp]
    mov     eax, DWORD PTR [rax]
    add     rsp, 56; 00000038H
    ret     0
main ENDP
```

```
passthrough$ =
; potential_dan
int const & pot
    mov
    ret
; potential_dan
int const & pot
```

```
main PROC
    mov
    ret
main ENDP
```

local constant but logically a global constant

```
const int& potential_dangler(const int& passthrough)
{
    return passthrough;
}

int main()
{
    const int local = 5;
    const int& reflocal = potential_dangler(local);
    return reflocal;
}
```

/Ox optimizations

```
passthrough$ = 8
; potential_dangler
int const & potential_dangler(int const &) PROC
    mov     QWORD PTR [rsp+8], rcx
    mov     rax, QWORD PTR passthrough$[rsp]
    ret     0
; potential_dangler
int const & potential_dangler(int const &) ENDP

local$ = 32
reflocal$ = 40
main PROC
$LN3:
    sub     rsp, 56; 00000038H
    mov     DWORD PTR local$[rsp], 5
    lea     rcx, QWORD PTR local$[rsp]
    ; potential_dangler
    call    int const & potential_dangler(int const &)
    mov     QWORD PTR reflocal$[rsp], rax
    mov     rax, QWORD PTR reflocal$[rsp]
    mov     eax, DWORD PTR [rax]
    add     rsp, 56; 00000038H
    ret     0
main ENDP
```

```
passthrough$ =
; potential_dan
int const & pot
    mov
    ret
; potential_dan
int const & pot
```

```
main PROC
    mov
    ret
main ENDP
```

constant expression i.e. logically a global constant

```
constexpr int return5()
{
    return 5;
}

const int& potential_dangler(const int& passthrough)
{
    return passthrough;
}

int main()
{
    const int& reflocal = potential_dangler(return5());
    return reflocal;
}
```

/Ox optimizations

```

int return5(void) PROC; return5, COMDAT
    mov     eax, 5
    ret     0
int return5(void) ENDP; return5

passthrough$ = 8
; potential_dangler
int const & potential_dangler(int const &) PROC
    mov     QWORD PTR [rsp+8], rcx
    mov     rax, QWORD PTR passthrough$[rsp]
    ret     0
; potential_dangler
int const & potential_dangler(int const &) ENDP

$T1 = 32
reflocal$ = 40
main     PROC
$LN3:
    sub     rsp, 56; 00000038H
    call    int return5(void); return5
    mov     DWORD PTR $T1[rsp], eax
    lea     rcx, QWORD PTR $T1[rsp]
    ; potential_dangler
    call    int const & potential_dangler(int const &)
    mov     QWORD PTR reflocal$[rsp], rax
    mov     rax, QWORD PTR reflocal$[rsp]
    mov     eax, DWORD PTR [rax]
    add     rsp, 56; 00000038H
    ret     0
main     ENDP

```

```

int return5(voi
    mov
    ret
int return5(voi

passthrough$ =
; potential_dan
int const & pot
    mov
    ret
; potential_dan
int const & pot

main     PROC
    mov
    ret
main     ENDP

```

an actual global

```
const int GLOBAL = 5;

const int& potential_dangler(const int& passthrough)
{
    return passthrough;
}

int main()
{
    const int& reflocal = potential_dangler(GLOBAL);
    return reflocal;
}
```

/Ox optimizations

```
int const GLOBAL DD 05H; GLOBAL

passthrough$ = 8
; potential_dangler
int const & potential_dangler(int const &) PROC
    mov     QWORD PTR [rsp+8], rcx
    mov     rax, QWORD PTR passthrough$[rsp]
    ret     0
; potential_dangler
int const & potential_dangler(int const &) ENDP

reflocal$ = 32
main PROC
$LN3:
    sub     rsp, 56; 00000038H
    lea     rcx, OFFSET FLAT:int const GLOBAL
    ; potential_dangler
    call    int const & potential_dangler(int const &)
    mov     QWORD PTR reflocal$[rsp], rax
    mov     rax, QWORD PTR reflocal$[rsp]
    mov     eax, DWORD PTR [rax]
    add     rsp, 56; 00000038H
    ret     0
main ENDP
```

```
passthrough$ =
; potential_dan
int const & pot
    mov
    ret
; potential_dan
int const & pot
```

```
main PROC
    mov
    ret
main ENDP
```

In all four cases, when optimizations (favor speed) is turned on, the Microsoft compiler produced the same **non dangling** code regardless of whether it was an actual global, a local constant, a temporary constant or a constant expression evaluation. This is also the same that GCC and Clang was generating. To the msvc compiler's credit, it not only detect functions that can potentially dangle but also executions that could as well. In all cases, it was a warning instead of an error. While the temporary constant and the constant expression evaluation is truly dangling when not optimized, it was not a compiler error. Further, the global example was incorrectly flagged as potentially dangling even though it new it was a global. Regardless the optimized compilation, fixed the dangling and removed the potentially dangling flag.

This proposal advocates standardizing an optimization that compiler's are already doing and have been doing before C++ got constexpr in the language. Fixing this type of dangling in this fashion is the best possible way because potentially invalid code becomes valid with no programmer intervention, it produces no errors, it is faster, uses less memory and produces smaller executable sizes. In short, the compiler/language already has all it needs to fix dangling constants. Compilers are already doing this but there is currently no verbiage in the standard that state that anonymous constants don't dangle because they are logically a global constant. Adopting this proposal ensures programmers do not have to fix something that was never dangling in the first place even though the current language makes it look like it is, needlessly.

Tooling Opportunities

There area a couple tooling opportunities especially with respect to the constexpr specifier.

- A command line and/or IDE tool could analyze the code for const , constexpr / LiteralType and constant-initialized and if the conditions matches automatically add the constexpr specifier for code reviewers.
- Another command line and/or IDE tool could strip constexpr specifier from any temporaries for programmers.

Combined they would form a constexpr toggle which wouldn't be all that much different from whitespace and special character toggles already found in many IDE(s).

Summary

The advantages to C++ with adopting this proposal is manifold.

- Safer
 - Eliminate dangling of what should be constants
 - Reduce immediate dangling when the instance is a constant
 - Reduce returning direct reference dangling when the instance is a constant

- Reduce returning indirect reference dangling when the instance is a constant and was provided as an argument
- Reduce indirect dangling that can occur in the body of a function
- Reduce uninitialized and delayed initialization errors
- Increases safety by avoiding data races.
- Simpler
 - Encourages the use of temporaries
 - Reduce lines of code
 - Reduce naming; fewer names to return dangle
 - Increases anonymously named `lvalues` and decreases `rvalues` in the code.
 - Reduce lines of code
 - Reduce naming; fewer names to return dangle
 - Make `constexpr` literals less surprising for new and old developers alike
 - Reduce the gap between `c++` and `c99` compound literals
 - Improve the potential contribution of `c++` 's dangling resolutions back to `c`
 - Make string literals and `c++` literals more consistent with one another
 - Taking a step closer to reducing undefined behavior in string literals
 - Simplify the language to match existing practice
 - Consequently, a “cleanup”, i.e. adoption of simpler, more general rules/guidelines
- Faster & More Memory Efficient
 - Reduce unnecessary heap allocations
 - Increase and improve upon the utilization of ROM and the benefits that entails

Frequently Asked Questions

What about locality of reference?

It is true that globals can be slower than locals because they are farther in memory from the code that uses them. So let me clarify, when I say `static storage duration`, I really mean **logically** `static storage duration`. If a type is a `PODType / TrivialType` or `LiteralType` then there is nothing preventing the compiler from copying the global to a local that is closer to the executing code. Rather, the compiler **must** ensure that the instance is always available; **effectively** `static storage duration`.

Consider this from an processor and assembly/machine language standpoint. A processor usually has instructions that works with memory. Whether that memory is ROM or is logically so because it is never written to by a program, then we have constants.


```
mov <register>,<memory>
```

A processor may also have specialized versions of common instructions where a constant value is taken as part of the instruction itself. This too is a constant. However, this constant is guaranteed closer to the code because it is physically a part of it.

```
mov <register>,<constant>  
mov <memory>,<constant>
```

What is more interesting is these two examples of constants have different value categories since the ROM version is addressable and the instruction only version, clearly, is not. It should also be noted that the later unnamed/unaddressable version physically can't dangle.

Won't this break a lot of existing code?

NO, if any. To the contrary, code that is broken is now fixed. Code that would be invalid is now valid, makes sense and can be rationally explained. Let me summarize:

This feature not only changes the point of destruction but also the point of construction. Instances that were of automatic storage duration, are now of static storage duration. Instances that were temporaries, are no longer temporaries. Surely, something must be broken! From the earlier section "Present", subsection "C Standard Compound Literals". Even the c++ standard recognized that there are other opportunities for constant initialization.

Working Draft, Standard for Programming Language C++ [7:6]

"6.9.3.2 Static initialization [basic.start.static]"

"3 An implementation is permitted to perform the initialization of a variable with static or thread storage duration as a static initialization even if such initialization is not required to be done statically ..."

So, what is the point? For the instances that would benefit from implicit constant initialization, there are currently NO guarantees as far as their lifetime and as such is indeterminate. With this portion of the proposal, a guarantee is given and as such that which was non-determinate becomes determinate.

It should also be noted that while this enhancement is applied implicitly, programmers have opted into this up to three times.

1. The programmer of the type must have provided a means for the type to be constructed at compile time likely by having a `constexpr` constructor.
2. The programmer of the variable or function parameter must have stated that they want a `const`.
3. The end programmer have `const-initialized` the variable or argument.

Having expressed constant requirements three times, it is pretty certain that the end programmer wanted a constant, even if it is anonymous.

Who would even use these features? Their isn't sufficient use to justify these changes.

Everyone ... Quite a bit, actually

Consider all the examples littered throughout our history, these are what gets fixed.

- dangling reported on normal use of the STL
- dangling examples reported in the C++ standard
- real world dangling reported in NAD, not a defect, reports

This doesn't even include the countless examples found in numerous articles comparing C++ with other nameless programming languages which would be fixed. However, the best proof can be found in our usage and other proposals.

C++ Core Guidelines ^[13]

F.16: For “in” parameters, pass cheaply-copied types by value and others by reference to const

In C++, we use `const` parameters alot. This is the first of three requirements of `implicit constant initialization`. What about the use of types that can be constructed at compile time?

- **C++20**: `std::pair`, `std::tuple`, `std::string`, `std::vector`
- **C++23**: `std::optional`, `std::variant`, `std::unique_ptr`

As there was sufficient use to justify making the constructors of any one of these listed above types to be `constexpr` than there would be sufficient use of the `implicit constant initialization` feature which would use them all as this satisfies its second and third requirement that the instances be constructable at compile time and `constant-initialized`.

Why not just use a static analyzer?

Typically a static analyzer doesn't fix code. Instead, it just produces warnings and errors. It is the programmer's responsibility to fix code by deciding whether the incident was a false positive or not and making the corresponding code changes. This proposal does fix some dangling but others go unresolved and unidentified. As such this proposal and static analyzers are complimentary. Combined this proposal can fix some dangling and a static analyzer could be used to identify what is remaining. As such those who still ask, "why not just use a static analyzer", might really be saying **this proposal's language enhancements might break their static analyzer**. To which I say, the standard dictates the analyzer, not the other way around. That is true for all tools. However, let's explore the potential impact of this proposal on static analyzers.

The `c++` language is complex. It stands to reason that our tools would have some degree of complexity, since they would need to take some subset of our language's rules into consideration. In any proposal, mine included, fixes to any dangling would result in potential dangling incidents becoming false positives between those identified by a static analyzer that overlap with said proposal. The false positives would join those that a static analyzer already has for not factoring existing language rules into consideration just as it would for any new language rules.

With `implicit constant initialization`, existing static analyzers would need to be enhanced to track the `const` ness of variables and parameters, whether or not the types of variables and parameters can be constructed at compile time and whether or not instances were constant-initialized. Until that happens, an existing dangling incident reported by static analyzer will just be a false positive. The total number of incidents remain the same and the programmer just need to recognize that it was a false positive which should be easy to do since constants are trivial and these rules are simple.

Can this even be implemented?

`c++` already provides static storage duration guarantee for instances of one type and allows it for many others.

- native string literals already have static storage duration
- compilers have been free for a long time to promote compile time constructed instances to have static storage duration
- any `LiteralType` instances that are constant-initialized are already prime candidates for compilers to promote to having static storage duration

Doesn't the `implicit constant initialization` feature make it harder for programmers to identify dangling and thus harder to teach?

If there was no dangling then there would be nothing to teach with respect to any dangling feature. Even the whole standard is not taught. So the more dangling we fix in the language, the less dangling that has to be taught to beginners. Consider the following example, does the new features make it easier or harder to identify dangling?

```
f({1,2}); // implicit constant initialization
// vs.
int i = 1;
f({i, 2}); // no implicit constant initialization
```

It is plain to see that `{1,2}` is constant-initialized as it is composed entirely of `LiteralType` (s). It is also plain to see that `{i,2}` is modifiable as its initialization statement is variable and dynamic due to the variable `i`. So the real questions are as follows:

- Is the first parameter to the function `f` `const`?
- Is the type of the first parameter to the function `f` a `LiteralType`?

The fact is some programmer had to have known the answer to both questions in order to have written `f({1,2})` in the first place. The case could be made that it would be nice to be able to use the `constexpr` keyword on temporary arguments, `f constexpr {1, 2}`, as this would allow those who don't write the code, such as code reviewers, to quickly validate the code. Even the programmer would benefit, some, if the code was copied. However, `constexpr` would mostly be superfluous, if the temporaries are just anonymously named variables feature is added. As such, `constexpr` should be optional. Consequently, any negative impact upon identifying and teaching dangling is negligible.

Yet, both `implicit` and `explicit` constant initialization feature, by itself, makes it easier to identify and teach dangling.

C++ Core Guidelines

F.43: Never (directly or indirectly) return a pointer or a reference to a local object ^[6:3]

...

Note *This applies only to non-static local variables. All static variables are (as their name indicates) statically allocated, so that pointers to them cannot dangle.* ^[6:4]

Instances that have static storage duration can't dangle. Currently in `C++`, instances that don't immediately dangle can still dangle later such as by returning. Using `static storage duration` short circuits the dangling identification process. An instance, once identified, doesn't

need to be factored into any additional dangling decision making process. Using more `static storage duration` speeds up the dangling identification process. This would also be of benefit to static analyzers that goes through a similar thought process.

Doesn't this make C++ harder to teach?

Until the day that all dangling gets fixed, any incremental fixes to dangling still would require programmers to be able to identify any remaining dangling and know how to fix it specific to the given scenario, as there are multiple solutions. Since dangling occurs even for things as simple as constants and immediate dangling is so naturally easy to produce, dangling resolution still have to be taught, even to beginners. As this proposal fixes these types of dangling, it makes teaching `C++` easier because it makes `C++` easier.

So, what do we teach now and what bearing does these teachings, the `C++` standard and this proposal have on one another.

C++ Core Guidelines

F.42: Return a `T*` to indicate a position (only) ^[12:3]

Note *Do not return a pointer to something that is not in the caller's scope; see F.43.* ^[6:5]

Returning references to something in the caller's scope is only natural. It is a part of our reference delegating programming model. A function when given a reference does not know how the instance was created and it doesn't care as long as it is good for the life of the function call (and beyond). Unfortunately, scoping temporary arguments to the statement instead of the containing block doesn't just create immediate dangling but it provides to functions references to instances that are near death. These instances are almost dead on arrival. Having the ability to return a reference to a caller's instance or a sub-instance thereof assumes, correctly, that reference from the caller's scope would still be alive after this function call. The fact that temporary rules shortened the life to the statement is at odds with what we teach. This proposal restores to some temporaries the lifetime of anonymously named constants which is not only natural but also consistent with what programmers already know. It is also in line with what we teach as was codified in the C++ Core Guidelines. One such is as follows:

C++ Core Guidelines

F.43: Never (directly or indirectly) return a pointer or a reference to a local object ^[6:6]

Reason *To avoid the crashes and data corruption that can result from the use of such a dangling pointer.* ^[6:7]

Other than turning some of these locals into globals, this proposal does not solve nor contradict this teaching. If anything, by cleaning up the simple dangling it makes the remaining more visible.

Further, what is proposed is easy to teach because we already teach it and it makes `C++` even easier to teach.

- We already teach that native string literals don't dangle because they have static storage duration. This proposal just extends the concept to constants, as expected. This increases good consistency and reduces a bifurcation that is currently taught.

All of this can be done without adding any new keywords or any new attributes. We just use constant concepts that beginners are already familiar with. In fact, we will be working in harmony with all that we already teach about globals in the `Core C++ Guidelines` ^[14].

- I.2: Avoid non-const global variables ^[12:4]
- I.22: Avoid complex initialization of global objects ^[12:5]
- F.15: Prefer simple and conventional ways of passing information ^[12:6]
- F.16: For "in" parameters, pass cheaply-copied types by value and others by reference to const ^[12:7]
- F.43: Never (directly or indirectly) return a pointer or a reference to a local object ^[6:8]
- R.5: Prefer scoped objects, don't heap-allocate unnecessarily ^[12:8]
- R.6: Avoid non-const global variables ^[12:9]
- CP.2: Avoid data races ^[12:10]
- CP.24: Think of a thread as a global container ^[12:11]
- CP.32: To share ownership between unrelated threads use `shared_ptr` ^[12:12]

How do these specifiers propagate?

These specifiers apply to the temporary immediately to the right of said specifier and to any child temporaries. It does not impact any parent or sibling temporaries. Consider these examples:

```

// all of the temporaries has the default temporary scope as
// specified by the module attribute otherwise statement scope
f({1, { {2, 3}, 4}, {5, 6} });
// only 4 has constinit scope
f({1, { {2, 3}, constinit 4}, {5, 6} });
// only {2, 3}, 2, 3 has constinit scope
f({1, { constinit {2, 3}, 4}, {5, 6} });
// only { {2, 3}, 4}, {2, 3}, 2, 3, 4 has constinit scope
f({1, constinit { {2, 3}, 4}, {5, 6} });
// all of the arguments have has constinit scope
f(constinit {1, { {2, 3}, 4}, {5, 6} });
// only 5 has constinit scope
f({1, { {2, 3}, 4}, {constinit 5, 6} });

```

References

1. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2658r0.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2658r0.html>) ↩
2. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2623r2.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2623r2.html>) ↩
3. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2266r3.html> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2266r3.html>) ↩ ↩
4. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2012r2.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2012r2.pdf>) ↩
5. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2644r0.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2644r0.pdf>) ↩
6. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object>
(<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#f43-never-directly-or-indirectly-return-a-pointer-or-a-reference-to-a-local-object>) ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩

7. <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf> (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf>) ↩ ↩ ↩ ↩ ↩ ↩ ↩
8. <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0936r0.pdf> (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0936r0.pdf>) ↩ ↩ ↩
9. <https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/constants> (<https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/constants>) ↩
10. <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3038.htm> (<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3038.htm>) ↩
11. <https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2917.pdf> (<https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2917.pdf>) ↩
12. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#glossary> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#glossary>) ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩ ↩
13. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-in> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rf-in>) ↩
14. <https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines> (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>) ↩